COS 350 Lecture 26

unix: diff, sdiff, tar

understanding C type declarations

exam / crib sheets

review

DIFF(1) User Commands DIFF(1)

NAME

diff - compare files line by line

SYNOPSIS

diff [OPTION]... FILE1 FILE2

DESCRIPTION

Compare files line by line.

-i Ignore case differences in file contents.
-E Ignore changes due to tab expansion.
-b Ignore changes in the amount of white space.
-w Ignore all white space.
-B Ignore changes whose lines are all blank.
-e Output an ed script.

file1

```
one
two
three
four
five
six
seven
eight
nine
ten
```

file2

```
one
1.5
two
three
four
FIVE
six
seven
eight
ten
```

sdiff -t file1 file2

```
one                     one
                      > 1.5
two                     two
three                   three
four                    four
five                  | FIVE
six                     six
seven                   seven
eight                   eight
nine                  <
ten                     ten
```

NAME

tar — The GNU version of the tar archiving utility

SYNOPSIS

tar function [options] [pathname ...]

DESCRIPTION

Tar stores and extracts files from a tape or disk archive.

The first argument to tar should be a function:

-c	create a new archive
-t	list the contents of an archive
-x	extract files from an archive

OTHER OPTIONS

-f ARCHIVE	use archive file or device ARCHIVE
-v	verbosely list files processed

EXAMPLES

Create archive.tar from files foo and bar.

```
tar -cf archive.tar foo bar
```

Create archive.tar of source_dir verbosely (showing each file included).

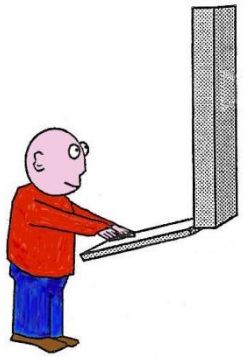
```
tar -cvf archive.tar source_dir
```

List all files in archive.tar verbosely (showing file details).

```
tar -tvf archive.tar
```

Extract all files from archive.tar.

```
tar -xf archive.tar
```



```
ls -R tartest
tar -cvf demo.tar tartest
od -vc demo.tar
mkdir untartest
cp demo.tar untartest
cd untartest
tar -xvf demo.tar

ls -l
gzip demo.tar
ls -l
```

The C type rule: *Declarations look like use*

Declaration	Use
`int a;`	`if (a > 1)`
`int *b;`	`a = *b;`
`int **c;`	`a = **c;`
`int d[50];`	`d[2] = d[0]+d[1];`
`int (*fn)(int, int);`	`a = (*fn)(2,6);` `a = fn(2,6);  // shorthand`

Operator Precedence	Associativity
`func() [] . -> expr++ expr--`	left-to-right
`* & + - ! ~ ++expr --expr (typecast) sizeof`	right-to-left
`* / %`	left-to-right
`+ -`	
`>> <<`	
`< > <= >=`	
`== !=`	
`&`	
`^`	
`|`	
`&&`	
`||`	
`?:`	right-to-left
`= += -= *= /= %= >>= <<= &= ^= |=`	right-to-left
`,`	left-to-right

Deciphering a complex declaration

Operator Precedence	Associativity
`func() []`	left-to-right
*	right-to-left

```
int a[40][10];
```

4 1 2 3

a is, an array of 40, array of 10, int

Deciphering a complex declaration

Operator Precedence	Associativity
`func() []`	left-to-right
`*`	right-to-left

```
int *b[10];
```

Deciphering a complex declaration

Operator Precedence	Associativity
`func() []`	left-to-right
`*`	right-to-left

```
int (*c)[10];
```

Deciphering a complex declaration

Operator Precedence	Associativity
`func() []`	left-to-right
`*`	right-to-left

```
int (*d[10])(char *argv[]);
```

Deciphering a complex declaration

Operator Precedence	Associativity
`func() []`	left-to-right
`*`	right-to-left

```
void (*signal(int sig, void (*func)(int)))(int);
```

Creating new names for types

```
char *s1;

typedef char *string;

string s2;
string argv[10];
```

Creating new names for types

```
#define __SLONGWORD_TYPE long int

#define __TIME_T_TYPE __SLONGWORD_TYPE

#define __STD_TYPE __extension__ typedef

__STD_TYPE __TIME_T_TYPE __time_t;      // Seconds since the Epoch.

typedef __time_t time_t;
```

Creating new names for types

```
void (*signal(int sig, void (*func)(int)))(int);
```

```
typedef void (*sighandler_t)(int);

sighandler_t signal(int sig, sighandler_t func);
```

Final Review

Review: Strings

Always make sure you have space for the string's length
+ 1 for the terminating 0.

Library Functions:

```
int strlen(char *s)
int strcmp(char *s1, char*s2)
char *strcpy(char *s1, char*s2)
char *strcat(char *s1, char*s2)
char *strdup(char *s)

char *strtok(char *s1, char *delimiters);
int atoi(char *s);

sprintf(char *buf, char *format, args ...);
```

Malloc

```
void *malloc(size_t size);

    char *buf = malloc( strlen(s1)+strlen(s2)+1 );

    int *array = malloc( n * sizeof(int) );

    struct list *node = malloc( sizeof(struct list) );
```

```
void free(void *ptr);

    free(buf);

    free(array);

    free(node);
```

Bitwise operations

&	bitwise AND	use with a mask for clearing bits
|	bitwise OR	use with a mask setting or selecting bits
^	bitise EXCLUSIVE OR	use with mask for switching bits from 0/1
~	ones complement	often use in constructing masks
<<	left shift bits, 0 bits fill from bottom	
>>	right shift bits, on signed types the sign bit is replicated	

Bitfield coding is the idea of storing multiple values in a single value using subsets of bits

Unbuffered File I/O (uses integer file descriptors)

```
int open(char *path, int flags);
int creat(char *path, mode_t mode);
size_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, void *buf, size_t count);
off_t lseek(int fd, off_t offset, int whence);
int close(int fd)
```

Buffered File I/O (uses pointers to FILE type)

```
FILE *fopen(char *path, char*mode);
int fgetc(FILE *stream);
char *fgets(char *s, int n, FILE *stream);
fscanf(FILE *stream, char *format, args ...);
fputc(int c, FILE *stream);
fputs(char *s, FILE *stream);
fprintf(FILE *stream, char *format, args ...);
fflush(FILE *stream);
fclose(FILE *stream);
```

Directories and file info

```
opendir(char *name)
readdir(DIR *dirp)
closedir(DIR *dirp)

scandir(char *dirname, struct dirent ***namelist, int (*filter)(), int (*cmp)())

int mkdir(char *pathname, mode_t mode)
int rmdir(char *pathname);

int link(char *oldpath, const char *newpath);
int unlink(char *pathname);
int symlink(char *oldpath, char *newpath);

int rename(char *oldpath, char *newpath);

int chdir(const char *path);
```

```
int stat(char *path, struct stat *buf);
int lstat(char *path, struct stat *buf);

char *getcwd(char *buf, size_t size);
```

Time

int gettimeofday(struct timeval *tv, struct timezone *tz);

```
struct timeval {
  time_t      tv_sec;     /* seconds */
  suseconds_t tv_usec;    /* and microseconds */
};
```

struct tm *localtime(time_t *timep);
char *ctime(time_t *timep);

sleep(int seconds);
usleep(useconds_t usec);
nanosleep(struct timespec *req, struct timespec *rem);

Signals

```
sighandler_t signal(int signum, sighandler_t handler);

int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);

int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Alarms & Timers

```
int alarm(unsigned int seconds);

int setitimer(int which, struct itimerval *new_value, struct itimerval *old_val);
int getitimer(int which, struct itimerval *curr_value);

int pause(void);
```

Terminal

```
extern FILE *stdin, *stdout, *stderr;
fd constants: STDIN_FILENO, STDOUT_FILENO, STDERR_FILENO
fd values: 0, 1, 2

int fd = open("/dev/tty");

int tcgetattr(int fd, struct termios *termios_p);

   [ ... ECHO (bit) ... ICANON (bit) ... VINTR (char) ... ]

int tcsetattr(int fd, int optional_actions, struct termios *termios_p);
```

Redirection and Pipes

```
int dup(int oldfd);
int dup2(int oldfd, int newfd);

int pipe(int pipefd[2]);
```

Processes

```
pid_t fork(void);

int execvp(const char *file, char *argv[]);
int execlp(const char *file, char *arg, ..., NULL);
    ...

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);

void exit(int status);
```

Threads

```
int pthread_create(pthread_t *thread, pthread_attr_t *attr,
                   void *(*start_routine)(void*),  void *arg);

void pthread_exit(void *ret_value_ptr);

int pthread_join(pthread_t thread, void **value_ptr);
```

Synchronization

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

    int pthread_mutex_lock(pthread_mutex_t *mutex);
    int pthread_mutex_trylock(pthread_mutex_t *mutex);
    int pthread_mutex_unlock(pthread_mutex_t *mutex);


pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

    int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
    int pthread_cond_signal(pthread_cond_t *cond);
```

There is a lot more to learn.